

Attack Calculus: A Typed, Evidence-Aware State-Transition Calculus for Cross-Domain Cybersecurity Reasoning

Formal Semantics, Evidence-Consistent Models, Robust Defensive Synthesis, and an AI-Agent Extension

Thor Thor

Independent Open-Source Researcher, THOR-SEC

ORCID: 0009-0001-6573-385X codethor@gmail.com

GitHub: <https://github.com/codethor0> <https://codethor0.github.io/thor-sec/>

This research was conducted independently on the author’s own time and is not sponsored by, affiliated with, or representative of any employer.

Preprint, October 2026. Released openly for community review.

Abstract

Cybersecurity frameworks classify adversary behavior, exchange threat intelligence, catalog defensive techniques, and generate attack graphs, yet incident reasoning still lacks a compact notation that states at once why an action occurs, how it occurs, which state enables it, which state it changes, and what evidence supports each claim. This paper proposes Attack Calculus Notation (ACN), a typed, evidence-aware state-transition calculus that connects analyst-readable incident reasoning to machine-checkable models. Each atomic action is an actor-labelled transition with explicit intent, mechanism, target, object, channel, valued guards, add and delete effects, and field-level evidence. Compact expressions denote finite trace languages and compile to a guarded predicate-transition model equivalent to typed STRIPS planning, which fixes the complexity of compromise checking. Evidence uncertainty defines a family of admissible models, over which ACN distinguishes possible from necessary compromise. Defensive controls are model transformations, and robust task-preserving synthesis selects the least-cost control bundle that eliminates every modeled prohibited outcome in every admissible model while keeping authorized-task utility above a floor in the worst admissible model. An AI-agent extension separates influence from authority and measures excess authority with a capability subsumption order. A complete worked example, reproduced by a published reference checker, shows a case where an analysis restricted to the most likely model selects a control that fails in an admissible alternative, and where attack-only minimization breaks a business task. The work is a formal specification and falsifiable research program; it claims no empirical superiority.

Keywords: attack modeling; state transitions; automated planning; evidence provenance; formal methods; defensive synthesis; least privilege; AI-agent security; incident reasoning

1. Introduction

Security analysis often begins with labels such as ransomware, phishing, identity compromise, prompt injection, supply-chain compromise, or process manipulation. These labels are useful for communication, but they do not determine execution logic. Two incidents with the same label may depend on different credentials, trust assumptions, channels, privilege transitions, reachable resources, or defensive failures. The central object of analysis is therefore not the attack name but the state transformation that makes the next action possible.

Attack Calculus begins from a simple premise: an attack is a sequence, branch, loop, or partially ordered set of state transformations. The useful unit of reasoning is an atomic transition with explicit prerequisites and effects. The notation is intended to be readable by incident responders and threat analysts, structured

enough for parsers, and constrained enough that unsupported prerequisites and contradictory effects are detected mechanically.

The problem is not a lack of formal models. Attacks have been described by preconditions and post-conditions since LAMBDA and the requires/provides model of JIGSAW [1, 2], and as state transitions for intrusion detection since STATL [3]. Alert correlation has linked attack steps through prerequisites and consequences [4]. Attack graphs have been generated by model checking and by logic programming [5, 6, 7], attack-defense trees provide goal decomposition with attacker-defender semantics [8], and the Meta Attack Language generates attack graphs from domain models [9, 10]. Automated planning has been used to generate attack courses of action and to simulate penetration testing [11, 12, 13]. ATT&CK supplies a behavioral taxonomy, D3FEND a countermeasure knowledge graph, and STIX an exchange format [20, 21, 22].

The narrower research question is whether a single analyst-facing intermediate representation can preserve causal execution logic and epistemic evidence while compiling to machine-checkable models that interoperate with these taxonomies. The working hypothesis is that such a representation makes disagreements inspectable, exposes missing prerequisites, keeps reconstruction from silently becoming fact, and connects incident reasoning to defensive synthesis without requiring analysts to author a full formal model.

ACN makes five proposed contributions: (1) a typed atomic transition with valued guards, additive and destructive effects, and field-level evidence; (2) a composition grammar with denotational trace semantics; (3) evidence-consistent model families with possible and necessary compromise; (4) robust task-preserving defensive synthesis with worst-case utility; and (5) a subsumption-aware measure of excess agent authority. A worked example in Section 8 exercises every definition with computed values, and a reference checker reproduces them.

2. Research Position and Relation to Prior Work

A credible calculus must state what it does not invent. Table 1 summarizes the position; this section gives the reasoning.

Precondition-effect attack languages. LAMBDA specified attacks by pre-condition, post-condition, scenario, detection, and verification [1]. JIGSAW modeled attacks through capabilities each step requires and provides [2]. Ning, Cui, and Reeves correlated alerts by matching consequences to prerequisites [4]. STATL describes intrusion scenarios as state transitions [3]. ACN inherits this precondition-effect view. Its differences are explicit delete effects, explicit negative values, field-level evidence status, and a defined trace semantics for composition.

Attack graphs and monotonicity. Model-checking attack graphs [5], scalable graph-based vulnerability analysis [6], and MulVAL [7] established automated attack-graph generation. The scalable approaches rely on a monotonicity assumption: an attacker never loses a capability once gained. That assumption is what makes them tractable, and it is also what ACN deliberately drops. Credential rotation, session termination, isolation, recovery, and defense impairment all remove facts. ACN therefore trades the polynomial guarantees of monotone analysis for the ability to represent these reversals, and states the resulting complexity in Section 7.2.

Automated planning. A ground ACN model with valued atoms, preconditions, add effects, and delete effects is a typed STRIPS planning task [14, 15], and model-relative compromise is plan existence. Planning has been applied to attack generation and penetration testing [11, 12, 13]. This connection is a strength rather than a threat to the contribution: it imports known complexity results [16], mature solvers, and a

Table 1: *Positioning of ACN relative to established work.*

System	Primary role	ACN position
LAMBDA, JIGSAW, STATL	Pre/post attack specification	ACN inherits pre/post transitions; adds delete effects, valued negatives, evidence, and trace semantics.
MulVAL and monotone attack graphs	Scalable attack-graph generation	ACN drops monotonicity to represent revocation and recovery, at a stated complexity cost.
STRIPS, PDDL, attack planning	Plan existence and generation	ACN models compile to typed STRIPS; ACN adds evidence families and task-preserving synthesis.
Attack-defense trees, MAL	Goal decomposition, domain DSL	ACN is trace and incident oriented; interoperation is preferable to duplication.
ATT&CK, D3FEND, STIX	Taxonomy, countermeasures, exchange	External mappings remain meta-data; guards, effects, and evidence are ACN semantics.
CaMeL, Progent, AuthGraph	Agent authorization and provenance	ACN models these controls as transformations in a shared cross-domain semantics.

precise vocabulary. ACN’s additions to plain planning are evidence objects and evidence-consistent model families, possible versus necessary compromise, the analyst-facing compact syntax, and synthesis that preserves authorized tasks across the family.

Concurrency and nets. The compiled object resembles a predicate/transition net [17], and the independence relation of Section 4.1 is the standard independence of Mazurkiewicz trace theory and partial-order reduction [18, 19], used here only as a conservative admission rule.

Defensive optimization. Network interdiction and resource-constrained optimization over attack graphs predate ACN [23, 24]. Neither state-transition modeling, attack-graph generation, planning-based attack analysis, nor finding a cut set is claimed as new. Structural defenses that change the model itself, such as service-graph morphing across security epochs [25], fit Definition 13 as model transformations.

AI-agent security. Prompt injection manipulates agent behavior through attacker-controlled content [26, 27]. Task Shield checks each action against the user’s task [28]. CaMeL separates trusted control flow from untrusted data with capability restrictions [29], and Progent enforces deterministic privilege policies on tool calls [30]. Vendor guidance recommends constraining the consequences of manipulation [31] and scoping agent identity, resources, and tools [32]. AuthGraph compares an authorization graph against execution provenance [33], reachability-based confinement recomputes capability restrictions after contamination [34], and SkillGuard treats agent skills as permission-bearing artifacts [35]. The confinement system of [34] is also named SkillGuard; the two are unrelated. ACN does not propose a new authorization primitive. It expresses influence, delegated authority, tool scope, resource reachability, and egress in the same transition semantics used for other domains, so that agent incidents and conventional incidents are analyzed with one model.

3. Formal Objects

3.1 Typed state, values, and invariants

Let $\mathcal{S}$ be an extensible set of sorts, such as Actor, Principal, Asset, Process, Credential, Token, Resource, Channel, Capability, Tool, Policy, Data, and PhysicalState. Each predicate symbol p has a signature $\text{sig}(p) = (s_1, \dots, s_n; V_p)$, where V_p is a finite value domain. A ground atom $p(v_1, \dots, v_n) = u$ is well typed exactly when each v_i has sort s_i and $u \in V_p$:

$$\Delta \vdash p(v_1, \dots, v_n) = u : \text{Atom} \iff \text{type}(v_i) = s_i \forall i \wedge u \in V_p, \quad (1)$$

where Δ is the declared type environment. Boolean predicates use $V_p = \{\text{true}, \text{false}\}$, and the shorthand $p(\vec{v})$ means $p(\vec{v}) = \text{true}$.

Let Atoms be the universe of well-typed ground atoms and Γ a set of domain invariants. A state Σ is a subset of Atoms that satisfies Γ . Γ always includes value exclusivity: for each predicate instance $p(\vec{v})$, at most one value is present. Negation-as-failure is not part of guard semantics. A guard that requires a negative condition uses an explicit value, such as $\text{AUTHZ}(A, j) = \text{denied}$, so that an unmodeled fact is never read as false. Uncertainty about which concrete state is correct is handled by the model families of Section 6.

The atom universe is partitioned into typed namespaces for knowledge, privilege, trust, control, reachability, persistence, data possession, integrity, and availability, written $N_K, \dots, N_{Av}$. The familiar nine-component view is a projection of the same set-valued state, not a second object:

$$\Sigma \equiv \langle K, P, Tr, C, R, Ps, Dr, I, Av \rangle, \quad K = \Sigma \cap N_K, \dots, Av = \Sigma \cap N_{Av}. \quad (2)$$

Domain extensions add disjoint namespaces, and Γ may constrain atoms across namespaces. Representative atoms include $\text{KNOW}(A, cred)$, $\text{PRIV}(A, h) = \text{admin}$, $\text{TRUST}(tok, svc) = \text{valid}$, $\text{REACH}(h_1, h_2)$, $\text{PERSIST}(x, h)$, $\text{POSSESS}(A, f)$, $\text{INTEGRITY}(r) = \text{valid}$, and $\text{AVAILABLE}(s)$.

3.2 Atomic transitions, principals, and binding

Definition 1 (Atomic ACN transition). An atomic transition τ is a tuple $\langle a, b, i, m, y, x, \chi, \text{Pre}, \text{Add}, \text{Del}, e \rangle$, where a is the executing principal; b is an optional influencing principal on whose behalf or under whose influence a acts; i is the intent; m the mechanism; y the target; x the acted-upon object; χ the channel; Pre, Add, and Del are finite sets of typed atoms; and e is an evidence object. A well-formed transition has $\text{Add}(\tau) \cap \text{Del}(\tau) = \emptyset$ and all fields type-correct.

The compact form is

$$a[\triangleleft b] \vdash [i \mid m] y(x) @ \chi : \text{Pre} \Rightarrow (\text{Add}, \text{Del}) \dot{:} e. \quad (3)$$

The influencing principal matters whenever the actor that executes an action is not the party responsible for it: an AI agent acting on injected content, a deputy misused by a caller, or a script run by a phished user. Keeping both principals avoids two errors: attributing an agent's action to the attacker as if the attacker held the agent's authority, and attributing it to the agent as if no attacker were involved.

Authoring templates may contain typed variables, but executable transitions are ground. A type-preserving substitution θ binds every variable consistently across all fields:

$$\text{Ground}(\tau, \theta) \text{ defined} \iff \theta \text{ is type-preserving and binds every variable of } \tau. \quad (4)$$

The turnstile denotes an asserted transition under stated conditions. It is not a claim that the expression is a deductive proof about the real world.

3.3 Enablement, application, and invariant timing

Definition 2 (Raw successor). For a well-formed ground transition, the candidate successor removes all delete effects and then inserts all add effects as one atomic update. The intermediate delete-only state is not observable and need not satisfy Γ :

$$\text{Next}(\Sigma, \tau) = (\Sigma \setminus \text{Del}(\tau)) \cup \text{Add}(\tau). \quad (5)$$

Definition 3 (Enablement). A transition is enabled exactly when every guard atom is present and the complete candidate successor satisfies Γ :

$$\text{Enabled}(\tau, \Sigma) \iff \text{Pre}(\tau) \subseteq \Sigma \wedge \text{Next}(\Sigma, \tau) \models \Gamma. \quad (6)$$

A candidate that violates Γ is not executable, and a validator reports it as an invariant violation.

Definition 4 (Application). Application is defined on the enabled domain and returns the validated successor:

$$\text{Apply}(\Sigma, \tau) = \text{Next}(\Sigma, \tau) \quad \text{for } \text{Enabled}(\tau, \Sigma). \quad (7)$$

Delete effects are essential. Credential rotation invalidates prior trust, isolation removes reachability, recovery removes persistence, and defense impairment removes a control's availability. A model that only accumulates facts cannot represent these reversals soundly.

3.4 Evidence objects

Definition 5 (Evidence object). An evidence object is

$$e = \langle \text{status}, \text{confidence}, \text{method}, \text{provenance}, \text{time}, \text{support} \rangle,$$

where $\text{status} \in \{\text{O}, \text{C}, \text{R}, \text{U}\}$ (observed, corroborated, reconstructed, unresolved); *confidence* is optional and requires a declared calibration *method*; *provenance* identifies source material; *time* is a timestamp or interval; and $\text{support} \subseteq \{\text{occurrence}, \text{guard}, \text{add_effect}, \text{delete_effect}, \text{ordering}, \text{mapping}\}$. Every transition carries an evidence object, and so does every atom of the initial state Σ_0 .

Field-level support allows evidence to establish that an event occurred without establishing the analyst's reconstructed guard. Attaching evidence to initial atoms closes a gap in which a trace could rest on an initial fact with no recorded basis. Evidence status is categorical metadata, not a probability, and does not change execution. ACN forbids deriving confidence by assigning fixed numbers to O, C, R, and U. Provenance records may be serialized with W3C PROV [37].

4. Composition Semantics

Compact expressions are for reading, but each must denote a precise set of finite executable traces. The core language provides identity, sequence, choice, finite repetition, and conservatively admitted concurrency:

$$E ::= \mathbb{K} \mid \tau \mid (E) \mid E \rightarrow E \mid E \parallel E \mid E \oplus E \mid E^*. \quad (8)$$

Postfix repetition binds most tightly, then sequence $\rightarrow$, then concurrency $\parallel$, then choice $\oplus$. Binary operators associate to the left. Identity contributes the empty trace and lets an optional stage be skipped without inventing a fictitious transition. Infinite or temporally recurring behavior is outside the core finite-trace semantics.

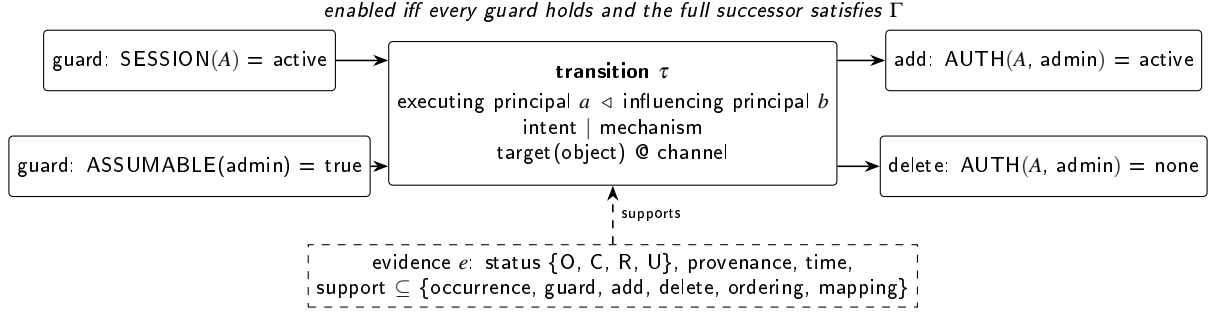


Figure 1: Atomic ACN transition. Guards are conjunctive and valued; add and delete effects are explicit and applied atomically; evidence records which parts of the transition are supported.

Definition 6 (Trace execution). $\text{End}(\Sigma, \pi)$ is the state reached by executing finite trace π from Σ , and is undefined if any step is not enabled where it occurs:

$$\text{End}(\Sigma, \varepsilon) = \Sigma; \quad \text{End}(\Sigma, \pi \cdot \tau) = \text{Apply}(\text{End}(\Sigma, \pi), \tau) \text{ if } \text{Enabled}(\tau, \text{End}(\Sigma, \pi)). \quad (9)$$

A trace is valid exactly when End is defined.

Definition 7 (Denotational trace semantics). $\mathcal{L}(E, \Sigma)$ is the set of finite traces licensed by E from Σ . State is threaded through sequence:

$$\mathcal{L}(\mathbb{K}, \Sigma) = \{\varepsilon\}; \quad \mathcal{L}(\tau, \Sigma) = \{\langle \tau \rangle\} \text{ if } \text{Enabled}(\tau, \Sigma), \text{ else } \emptyset, \quad (10)$$

$$\mathcal{L}(E \rightarrow F, \Sigma) = \{\pi \cdot \rho : \pi \in \mathcal{L}(E, \Sigma), \rho \in \mathcal{L}(F, \text{End}(\Sigma, \pi))\}, \quad (11)$$

$$\mathcal{L}(E \oplus F, \Sigma) = \mathcal{L}(E, \Sigma) \cup \mathcal{L}(F, \Sigma), \quad (12)$$

$$\mathcal{L}(E^*, \Sigma) = \bigcup_{n \geq 0} \mathcal{L}(E^n, \Sigma), \quad E^0 = \mathbb{K}, \quad E^{n+1} = E^n \rightarrow E, \quad (13)$$

$$\mathcal{L}(E \parallel F, \Sigma) = \{\sigma \in \text{Sh}(\pi, \rho) : \pi \in \mathcal{L}(E, \Sigma), \rho \in \mathcal{L}(F, \Sigma), \\ \text{End}(\Sigma, \sigma) \text{ defined, } \text{Ind}(\sigma, \pi, \rho)\}, \quad (14)$$

where $\text{Sh}(\pi, \rho)$ is the set of order-preserving interleavings and $\text{Ind}(\sigma, \pi, \rho)$ holds when, at every prefix state of σ , every pair of a pending step of π and a pending step of ρ that are co-enabled is independent in the sense of Equation (15).

Bounded repetition $E^{\leq k} = \bigcup_{n \leq k} E^n$ is a derived form used in Section 9.1.

4.1 Conservative concurrency

For ground transitions τ and ν , independence at Σ requires both to be enabled, each to remain enabled after the other, and both orders to reach the same state:

$$\tau \perp_{\Sigma} \nu \iff \text{Enabled}(\tau, \Sigma) \wedge \text{Enabled}(\nu, \Sigma) \wedge \text{Enabled}(\tau, \text{Apply}(\Sigma, \nu)) \wedge \text{Enabled}(\nu, \text{Apply}(\Sigma, \tau)) \\ \wedge \text{Apply}(\text{Apply}(\Sigma, \tau), \nu) = \text{Apply}(\text{Apply}(\Sigma, \nu), \tau). \quad (15)$$

This is the independence relation of trace theory and partial-order reduction [18, 19], used as a sufficient condition for admitting $\parallel$. It rejects conflicting writes and guard destruction without attempting to admit every safe partial order. An expression $E \parallel F$ is well formed only when independence can be established wherever concurrency is claimed.

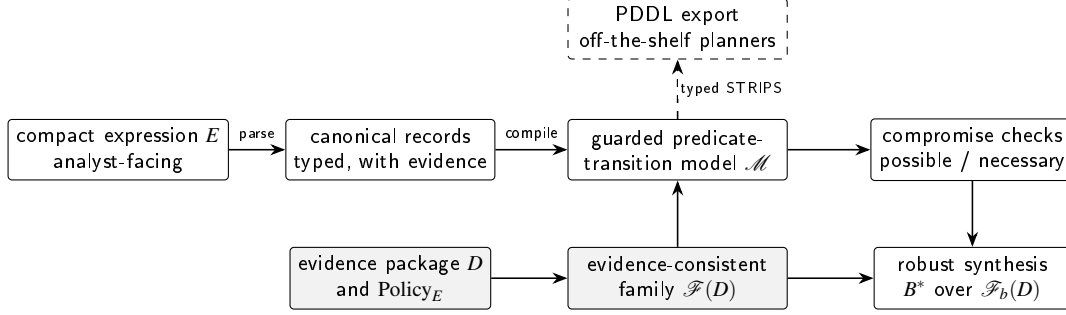


Figure 2: From analyst expression to analysis. Compilation must preserve the trace language (Proposition 1). Evidence determines which models are admissible; analyses quantify over that family. Because the compiled model is a typed STRIPS task, it can be exported to existing planners.

5. Canonical Model and Compiler Obligation

An ordinary directed graph cannot preserve ACN semantics, because a transition may require several valued atoms at once and may delete facts that later steps need. The compiled object is a guarded predicate-transition model in the sense of predicate/transition nets [17], and equivalently a typed STRIPS task [14].

Definition 8 (Guarded predicate-transition model). $\mathcal{M} = \langle \text{Atoms}, T, I_{\text{pre}}, I_{\text{add}}, I_{\text{del}}, \Gamma \rangle$, where T is a finite set of ground transitions, $I_{\text{pre}} \subseteq \text{Atoms} \times T$ is precondition incidence, and $I_{\text{add}}, I_{\text{del}} \subseteq T \times \text{Atoms}$ encode effects. Transitions fire under Equation (6) and update under Equation (7).

Compromise is therefore executable reachability, not topological reachability. Shortest-path and cut algorithms are valid only after a semantics-preserving reduction, or for a restricted class, such as monotone models without invariants, in which their assumptions hold.

A canonical record minimally contains transition identity, executing and influencing principals, intent, mechanism, typed target, object, and channel, variable declarations or a ground instance, valued guards, add and delete effects, evidence, model namespace, and optional mappings to ATT&CK, CAPEC, D3FEND, STIX, CWE, CVE, or ATLAS. Mappings remain metadata. Appendix Appendix B. gives an example, and Figure 2 shows the pipeline.

Proposition 1 (Compiler correctness obligation). A conformant compiler $\mathcal{C}$ satisfies $\text{Traces}(\mathcal{C}(E), \Sigma_0) = \mathcal{L}(E, \Sigma_0)$ for every well-formed expression E and initial state Σ_0 : it neither omits a licensed trace nor introduces an unlicensed one.

This is a specification that an implementation must discharge, not a theorem about all compilers. The proof strategy is structural induction on E : the atomic and identity cases follow from Equations (7) and (10); sequence from state threading; choice from union; repetition from the inductive unfolding; and concurrency from the restriction in Equation (14). A reference implementation should make the obligation executable through round-trip tests and generated counterexamples.

6. Evidence-Consistent Model Families

Incident reconstruction rarely yields one certain model. Evidence may establish that an event occurred while leaving a guard ambiguous, or establish an initial fact without its origin. ACN therefore separates system semantics from epistemic admissibility.

Definition 9 (Evidence-admission policy). Policy_E is a declared rule set mapping an evidence object and the model field it supports to one of $\{\text{required}, \text{admissible}, \text{excluded}\}$. A policy may use status, provenance, support scope, time, and confidence (only when a method is declared). ACN does not prescribe a universal policy; the chosen policy is part of the research record.

Definition 10 (Evidence-consistent family). For an evidence package D , the family contains every model and initial state whose atoms, initial facts, transitions, guards, effects, bindings, and temporal relations satisfy the constraints required or permitted by Policy_E and all type, domain, and invariant constraints:

$$\mathcal{F}(D) = \{(\mathcal{M}, \Sigma_0) : \text{EvidenceConsistent}(\mathcal{M}, \Sigma_0, D, \text{Policy}_E) \wedge \text{DomainConsistent}(\mathcal{M}, \Sigma_0)\}. \quad (16)$$

Uncertainty may concern an initial fact, a guard value, an effect, a binding, or temporal order, not only the presence of an edge. Evidence status changes which models are admissible, never transition probability or execution.

7. Compromise, Complexity, and Robust Defensive Synthesis

7.1 Prohibited goals, compromise, and modal compromise

A prohibited goal g is a finite set of well-typed atoms whose joint satisfaction constitutes the modeled failure; G is a finite set of goals.

Definition 11 (Model-relative compromise).

$$\text{Compromise}(\mathcal{M}, \Sigma_0, g) \iff \exists \pi \in T^* : \text{End}(\Sigma_0, \pi) \text{ defined} \wedge \text{End}(\Sigma_0, \pi) \models g. \quad (17)$$

Traces range over all transitions of $\mathcal{M}$, including environment and legitimate-user transitions, because an attacker can benefit from them. This is the conservative choice. Incident replay may instead restrict traces to $\mathcal{L}(E, \Sigma_0)$ for a reconstructed expression E ; the restriction must be declared. The qualifier model-relative is mandatory: ACN establishes reachability or non-reachability only inside a declared model scope.

Definition 12 (Possible and necessary compromise). Over an evidence-consistent family,

$$\text{PossiblyCompromised}(D, g) \iff \exists (\mathcal{M}, \Sigma_0) \in \mathcal{F}(D) : \text{Compromise}(\mathcal{M}, \Sigma_0, g), \quad (18)$$

$$\text{NecessarilyCompromised}(D, g) \iff \forall (\mathcal{M}, \Sigma_0) \in \mathcal{F}(D) : \text{Compromise}(\mathcal{M}, \Sigma_0, g). \quad (19)$$

The distinction answers two different analyst questions. Necessary compromise is what the evidence forces; possible compromise is what a defender must still rule out. Conformance Requirement 3 in Section 12 depends on keeping the two apart.

7.2 Complexity

Because a ground ACN model is a typed STRIPS task, deciding Compromise for one model is plan existence. For propositional STRIPS this is PSPACE-complete in general and NP-complete when plan length is polynomially bounded [16]; these bounds are measured in the size of the ground model, which typed grounding can make exponentially larger than the compact expression. With finite domains the state space is finite, so reachability is decidable without any trace horizon; a horizon L is needed only to enumerate $\mathcal{L}(E^*, \Sigma)$, and if one is imposed the guarantee weakens to Compromise_L , compromise within L steps, which must then be stated. The synthesis problem below contains non-reachability as the special case of an empty control library, so it is PSPACE-hard in general. Under a polynomial horizon its decision version lies

in Σ_2^p (guess a bundle, then verify safety in every admissible model and task achievability), and it remains NP-hard because weighted hitting set and network interdiction embed as restricted cases [23, 24]. Monotone attack-graph tools avoid these bounds by assuming facts are never deleted [6, 7]; ACN pays the cost in exchange for representing revocation and recovery.

7.3 Controls as transformations

Definition 13 (Control). A control κ is a deterministic transformation $\kappa(\mathcal{M}, \Sigma_0) = (\mathcal{M}^\kappa, \Sigma_0^\kappa)$. It may disable a transition, strengthen a guard, revoke trust, narrow authority, constrain a channel, block an effect, restore state, change an invariant, or restructure the model.

A bundle $B = \langle \kappa_1, \dots, \kappa_m \rangle$ declares an application order and passes a conflict check:

$$\text{ApplyControls}(B, (\mathcal{M}, \Sigma_0)) = \kappa_m \circ \dots \circ \kappa_1(\mathcal{M}, \Sigma_0) = (\mathcal{M}^B, \Sigma_0^B). \quad (20)$$

If the controls commute on the relevant models, a canonical order may be used. The cost $J(B)$ is any declared bundle-level function and need not be additive.

7.4 Robust task-preserving synthesis

Pure attack-path minimization can select a control that destroys the business function. An authorized task is $q = (A_q, g_q)$: a set of authorized principals and a task goal. Let T_{A_q} be the transitions whose executing principal is in A_q or is an environment principal.

$$U_Q(B; \mathcal{M}, \Sigma_0) = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}[\exists \pi \in T_{A_q}^* : \text{End}(\Sigma_0^B, \pi) \models g_q \text{ in } \mathcal{M}^B]. \quad (21)$$

Utility is defined per model because a control can break a task in one admissible model and not in another; weighted tasks are a direct generalization. Given a finite bounded family $\mathcal{F}_b(D) \subseteq \mathcal{F}(D)$, control bundles $\mathcal{B}$, goals G , and utility floor η , the robust synthesis problem is

$$\begin{aligned} B^* &\in \arg \min_{B \in \mathcal{B}} J(B) \\ \text{s.t. } &\forall (\mathcal{M}, \Sigma_0) \in \mathcal{F}_b(D), \forall g \in G : \neg \text{Compromise}(\mathcal{M}^B, \Sigma_0^B, g), \\ &\min_{(\mathcal{M}, \Sigma_0) \in \mathcal{F}_b(D)} U_Q(B; \mathcal{M}, \Sigma_0) \geq \eta, \end{aligned} \quad (22)$$

with $B^* = \perp$ when no bundle is feasible. The utility constraint is worst-case over the same family as the safety constraint, so that a bundle cannot be accepted on the strength of a model in which it happens to preserve tasks.

Exact bounded profile. Exact claims require a finite control library and bundle set, finite typed domains, finite task and goal suites, and a finite set of evidence alternatives. If h independent binary evidence hypotheses are admitted, the family has at most 2^h members before domain consistency prunes it; a commuting k -control library has at most 2^k subsets. These are worst-case enumeration bounds, not efficiency guarantees. Larger instances may use counterexample-guided synthesis: solve against a working subset of models, search $\mathcal{F}_b(D)$ for a violating model and trace, add it, and repeat. Results based on sampling or heuristic pruning must be labeled approximate and cannot support the universal quantifier in Equation (22).

8. Worked Example

This section exercises every definition on one small model. Every value is computed by the reference checker `acn_checks.py`, published with this paper. The example is a synthetic identity incident with no exploit content.

Evidence and family. Logs show that attacker A authenticated as user U with a stolen password, that multi-factor authentication was not enforced for U , and that A later read a protected database. How A obtained administrative authority is unresolved: the evidence admits role assumption from U 's session (model $\mathcal{M}_1$) and use of an administrative key exposed in a repository (model $\mathcal{M}_2$). Under a policy that admits reconstructed escalation mechanisms, $\mathcal{F}_b(D) = \{\mathcal{M}_1, \mathcal{M}_2\}$. The two differ in one initial fact, $\text{KEY}(\text{admin}) = \text{exposed}$ with $\text{KNOW}(A, \text{key})$, and one transition, t_{2b} .

Transitions. With explicit values throughout:

$$\begin{aligned}
 t_1 : A \vdash [\text{IA} \mid \text{VALID_CREDS}] \text{IdP}(U) : \text{KNOW}(A, \text{cred}_U) \wedge \text{MFA}(U) = \text{off} \wedge \text{SESSION}(A) = \text{none} \\
 \quad \Rightarrow \text{SESSION}(A) = \text{active} \\
 t_2 : A \vdash [\text{PE} \mid \text{ASSUME}] \text{IAM}(\text{admin}) : \text{SESSION}(A) = \text{active} \wedge \text{ASSUMABLE}(\text{admin}) = \text{true} \\
 \quad \Rightarrow \text{ROLESESS}(A) = \text{active}, \text{AUTH}(A, \text{admin}) = \text{active} \\
 t_{2b} : A \vdash [\text{PE} \mid \text{KEY}] \text{IAM}(\text{admin}) : \text{KEY}(\text{admin}) = \text{exposed} \wedge \text{KNOW}(A, \text{key}) \\
 \quad \Rightarrow \text{AUTH}(A, \text{admin}) = \text{active} \quad (\mathcal{M}_2 \text{ only}) \\
 t_3 : A \vdash [\text{COL} \mid \text{READ}] \text{DB}(\text{records}) : \text{AUTH}(A, \text{admin}) = \text{active} \wedge \text{READGATE} = \text{open} \\
 \quad \Rightarrow \text{KNOW}(A, \text{records}) \\
 t_5 : A \vdash [\text{STL} \mid \text{LOGOUT}] \text{IdP}(U) : \text{SESSION}(A) = \text{active} \Rightarrow \text{SESSION}(A) = \text{none}
 \end{aligned}$$

Each value update deletes the previous value of the same atom. The invariant Γ adds value exclusivity and $\text{ROLESESS}(x) = \text{active} \Rightarrow \text{SESSION}(x) = \text{active}$. Legitimate transitions let U log in (with MFA if enforced, using a registered device) and assume the admin role, and let the backup service read the database when $\text{READGATE} = \text{open}$.

Semantics in action. In Σ_0 of $\mathcal{M}_1$, t_1 is enabled. After t_1 and t_2 , t_5 is *not* enabled: its guard holds, but the successor would leave a role session without a login session, violating Γ . This is the invariant-timing rule of Equation (6) rejecting a state that a guard-only check would accept. The trace $t_1 t_2 t_3$ is valid. After t_2 , the attacker read t_3 and the backup read are independent under Equation (15), so they may be composed with $\parallel$. After t_1 alone, t_2 and t_5 are both enabled but not independent: logging out disables role assumption.

Compromise. With goal $g = \{\text{KNOW}(A, \text{records})\}$, both models are compromised, with shortest witnesses $t_1 t_2 t_3$ in $\mathcal{M}_1$ and $t_{2b} t_3$ in $\mathcal{M}_2$. The incident is therefore necessarily compromised. After control k_1 alone it is possibly but not necessarily compromised.

Controls and tasks. The control library is k_1 enforce MFA (cost 2), k_2 revoke role assumability (cost 1), k_3 rotate and vault the exposed key (cost 1), and k_4 require break-glass approval for every database read (cost 4). The controls set independent initial values, so they commute. The task suite is q_1 : U can obtain admin authority for maintenance, and q_2 : the backup service completes its read. Table 2 lists representative bundles out of all 16 evaluated.

Table 2: Selected bundles from the worked example. All 16 subsets are evaluated exactly by the reference checker.

Bundle B	$J(B)$	Safe in $\mathcal{M}_1$	Safe in $\mathcal{M}_2$	$\min U_Q$	Note
$\emptyset$	0	no	no	1.0	necessarily compromised
$\{k_1\}$	2	yes	no	1.0	optimal if only $\mathcal{M}_1$ is considered
$\{k_2\}$	1	yes	no	0.5	breaks q_1
$\{k_3\}$	1	no	no	1.0	
$\{k_4\}$	4	yes	yes	0.5	breaks q_2
$\{k_2, k_3\}$	2	yes	yes	0.5	attack-only optimum
$\{k_1, k_3\}$	3	yes	yes	1.0	robust optimum at $\eta = 1$

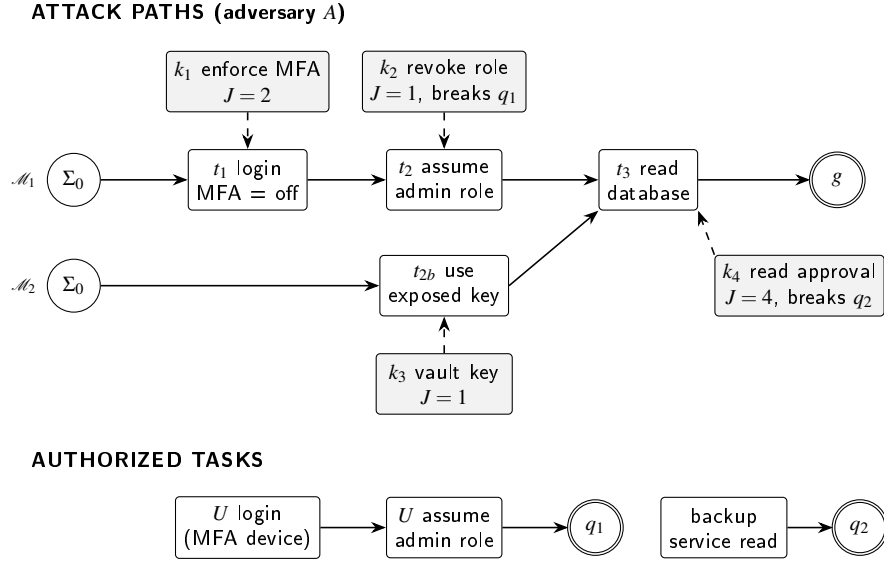


Figure 3: Worked example. Solid arrows are transitions; dashed arrows show which transition each control disables. k_2 also blocks the legitimate role assumption in q_1 , and k_4 also blocks the backup read in q_2 . $\{k_1, k_3\}$ blocks every attack path in both admissible models and preserves both tasks; $\{k_1\}$ alone leaves the $\mathcal{M}_2$ path open; $\{k_2, k_3\}$ is cheaper but breaks q_1 .

Results. Three findings follow, each computed exactly.

1. *Robust synthesis.* With $\eta = 1$, $B^* = \{k_1, k_3\}$ at cost 3.
2. *Single-model analysis fails.* Restricting the analysis to $\mathcal{M}_1$, the most direct reading of the logs, selects $\{k_1\}$ at cost 2. That bundle leaves $\mathcal{M}_2$ compromised: the attacker escalates through the exposed key. Quantifying over the family is what exposes the gap.
3. *Attack-only minimization breaks the business.* Ignoring utility ($\eta = 0$) selects $\{k_2, k_3\}$ at cost 2, which blocks both attack paths but removes U 's legitimate maintenance access, so $\min U_Q = 0.5$. The same bundle is the correct answer if the operator declares $\eta = 0.5$, which makes the business tradeoff an explicit, recorded decision rather than a side effect of the optimizer.

Excess authority. Section 10.1 is exercised on an agent holding

$$H = \{\text{read}(/data/*), \text{send}(*@corp), \text{delete}(/tmp/*), \text{share}(/data/report)\}$$

with weights 3, 2, 4, 1, and a task with two incomparable minimal realizations,

$$R_1 = \{\text{read}(/data/report), \text{send}(\text{boss@corp})\}, \quad R_2 = \{\text{read}(/data/report), \text{share}(/data/report)\}.$$

Plain set inclusion reports both realizations infeasible, because neither $R_1 \subseteq H$ nor $R_2 \subseteq H$, although H clearly covers both. With the subsumption order both are feasible, the excess is 5 for R_1 and 6 for R_2 , and $[EA_{\min}, EA_{\max}] = [5, 6]$.

9. Cross-Domain Stress Tests

These examples test whether one typed guard and effect semantics covers materially different domains. They are semantic stress tests, not empirical demonstrations, and they avoid exploit-specific detail.

9.1 Mobile reachability: Manic

ThreatFabric’s August 2026 analysis of the Manic Android malware reports that when direct command-and-control connectivity is unavailable, infected devices queue encrypted data and relay it through nearby infected devices over Wi-Fi Direct, Bluetooth RFCOMM, or Bluetooth Low Energy, with a default limit of four relay hops carried in the relay metadata [38]. The graph-theoretic observation is modest:

$$\neg \text{Direct}(v, C_2) \not\Rightarrow \neg \text{Reachable}(v, C_2). \quad (23)$$

The state that makes it operational is queued data, a nearby infected peer, an alternate radio path, and a remaining hop budget. One relay hop from device v to peer p is

$$\begin{aligned} w \vdash [\text{EXF} \mid \text{RELAY}] \text{PEER}(d) @ \text{RADIO} : \\ \text{HOLDS}(v, d) \wedge \text{NEAR}(v, p) \wedge \text{INFECTED}(p) \wedge \text{HOPS}(d) = k \wedge k < 4 \\ \Rightarrow \text{HOLDS}(p, d), \text{HOPS}(d) = k+1 \vdash R, \end{aligned} \quad (24)$$

where w denotes the malware instance, so as not to overload the model symbol $\mathcal{M}$, and the update to HOPS deletes the previous value. Exfiltration completes with an upload transition guarded by $\text{HOLDS}(p, d) \wedge \text{ONLINE}(p)$. The full behavior is the bounded expression $\text{RELAY}^{\leq 4} \rightarrow \text{UPLOAD}$. The example shows why removing direct Internet reachability from one device does not remove every modeled path, and why the hop budget belongs in the state rather than in prose.

9.2 Ransomware as a trace schema

A ransomware operation is not one universal linear theorem. With IA initial access, EXE execution, PE privilege escalation, PER persistence, DIS discovery, CA credential access, LM lateral movement, DIMP defense impairment, COL collection, and EXF exfiltration,

$$R = \text{IA} \rightarrow \text{EXE} \rightarrow (\text{PE} \oplus \mathbb{K}) \rightarrow \text{PER}^* \rightarrow \text{DIS} \rightarrow (\text{CA} \rightarrow \text{LM})^* \rightarrow \text{DIMP}^* \rightarrow (\text{COL} \rightarrow \text{EXF})^* \rightarrow \text{IMP}_{\text{encrypt}} \quad (25)$$

is a trace schema, not a definition. Each realized label expands to typed transitions with concrete guards and effects. The DIMP label corresponds to the Defense Impairment tactic that ATT&CK v19 split from the former Defense Evasion tactic, alongside Stealth [20]; such labels remain external metadata.

9.3 API authorization failure

Absence, denial, and non-enforcement must be distinguished. A broken object-level authorization transition states all three explicitly:

$$\begin{aligned}
& A \vdash [\text{COL} \mid \text{ACCESS}] \text{API}(r_j)@HTTPS : \\
& \quad \text{AUTHN}(A) \wedge \text{AUTHZ}(A, j) = \text{denied} \wedge \text{ENFORCED}(\text{API}, j) = \text{false} \\
& \Rightarrow \text{KNOW}(A, r_j) \dot{:} R.
\end{aligned} \tag{26}$$

A secure API with $\text{ENFORCED}(\text{API}, j) = \text{true}$ rejects the same denied decision. Not being authorized is not sufficient for disclosure, and unknown is never read as false.

9.4 Cloud identity

Cloud incidents often move through delegated identity rather than host traversal:

$$\begin{aligned}
& A \vdash [\text{PE} \mid \text{ABUSE}] \text{IAM}(r)@STS : \\
& \quad \text{TOKEN}(A, t) = \text{valid} \wedge \text{ASSUME_ALLOWED}(t, r) \\
& \Rightarrow \text{AUTH}(A, r) = \text{active} \dot{:} R.
\end{aligned} \tag{27}$$

The ACN effect is the authority state that changed; ATT&CK labels remain classification metadata.

9.5 Cyber-physical process state

Industrial systems need process-state predicates and external safety invariants:

$$\begin{aligned}
& A \vdash [\text{IPC} \mid \text{STATE}] \text{PLC}(sp)@OT : \\
& \quad \text{CONTROL}(A, \text{EWS}) \wedge \text{WRITE_ALLOWED}(\text{EWS}, \text{PLC}) \wedge \text{INTERLOCK}(\text{PLC}) = \text{bypassed} \\
& \Rightarrow \text{MODE}(\text{PLC}) = \text{unsafe} \dot{:} R.
\end{aligned} \tag{28}$$

This is a discrete security abstraction, not a plant model. Γ may reference invariants verified by continuous-control or hybrid-system analysis, and a hardware interlock is a control that prevents the unsafe successor.

10. AI-Agent Extension

AI agents stress the calculus because security-relevant state is neither purely host based nor purely identity based. The model must separate attacker-influenced context from deterministic policy, model-selected actions from authorized effects, and user authority from authority delegated to an agent. Existing work addresses each of these [28, 29, 30, 33, 34, 35]; ACN's role is to express them in one transition semantics.

$$\Sigma^{\text{AI}} = \langle \Sigma; X, H, \mathcal{O}, \mathcal{R}, \Pi, \mathcal{E} \rangle, \tag{29}$$

where X is context and decision state, H the agent's enforceable capability set, $\mathcal{O}$ the enabled tool operations, $\mathcal{R}$ the reachable resources, Π deterministic policy enforced outside generative inference, and $\mathcal{E}$ the permitted external effects. Π changes only through an explicit policy transition or control. The agent appears in transitions as executing principal ag , with the attacker as influencing principal where influence is modeled (Definition 1).

10.1 Capabilities and subsumption

A capability is parameterized: $c = \langle op, scope, constraints, effect, validity \rangle$. Because scopes nest, authority comparison needs an order rather than set equality. Define $c \sqsubseteq c'$ (c' covers c) when both have the same operation and effect class, $scope(c) \subseteq scope(c')$, every argument satisfying $constraints(c)$ satisfies $constraints(c')$, and $validity(c) \subseteq validity(c')$. Lift it to sets:

$$R \sqsubseteq H \iff \forall c \in R \exists c' \in H : c \sqsubseteq c', \quad \text{Used}(H, R) = \{c' \in H : \exists c \in R, c \sqsubseteq c'\}. \quad (30)$$

The delegation invariant is $H_{\text{agent}}(t) \sqsubseteq H_{\text{deleg}}(user, ctx, t)$, where $H_{\text{deleg}} = \text{Delegable}(\Pi_t, user, ctx, t)$. Plain set inclusion is wrong here: an agent holding $\text{read}(/data/*)$ covers a required $\text{read}(/data/report)$, but $\{\text{read}(/data/report)\} \not\subseteq \{\text{read}(/data/*)\}$. Section 8 shows inclusion reporting a covered task as infeasible.

10.2 Minimal sufficient authority and excess authority

$\text{Req}(q, t)$ is the family of minimal capability sets sufficient for authorized task q at time t under the declared platform, policy, and workflow model, minimal with respect to $\sqsubseteq$. The feasible realizations are

$$\text{Req}_H(q, t) = \{R \in \text{Req}(q, t) : R \sqsubseteq H_{\text{agent}}(t)\}. \quad (31)$$

If an authorized plan p is selected with requirement R_p , the plan-relative Authority Gap is the set of held capabilities that the plan never uses:

$$\Delta H_p(t) = H_{\text{agent}}(t) \setminus \text{Used}(H_{\text{agent}}(t), R_p). \quad (32)$$

When several incomparable minimal realizations exist and no plan has been selected, ACN reports an interval rather than inventing a canonical requirement. With a declared nonnegative exposure weight w and documented calibration,

$$\text{EA}_{\min}(q, t) = \min_{R \in \text{Req}_H(q, t)} \sum_{c' \in H \setminus \text{Used}(H, R)} w(c'), \quad \text{EA}_{\max}(q, t) = \max_{R \in \text{Req}_H(q, t)} \sum_{c' \in H \setminus \text{Used}(H, R)} w(c'). \quad (33)$$

Both are undefined when $\text{Req}_H(q, t) = \emptyset$, meaning current authority is insufficient. A runtime policy that must choose one realization before planning declares its own deterministic selection rule, which becomes part of Π and of the research record. The measure counts unused capabilities. Over-breadth of a used capability, such as holding $\text{read}(/data/*)$ to read one file, requires a declared scope measure and is left as an explicit extension. $\text{EA}_{\min}$ and $\text{EA}_{\max}$ measure removable excess authority under stated weights; they are not breach probabilities.

10.3 From influence to a prohibited effect

Retrieved data is not assumed to become a system instruction. The weaker security-relevant predicate is that attacker-controlled content influences decision state. Unless telemetry supports the causal link, the step is reconstructed:

$$\begin{aligned} A \vdash [\text{AIS} \mid \text{INJECT}] \text{ctx}(X) @ \text{CONTEXT} : \\ \text{RETRIEVED}(ag, d_m) \wedge \text{ATTACKER_CTRL}(d_m) \wedge \text{ACTIVE}(ag, d_m) \\ \Rightarrow \text{INFL}(A, X_{ag}) \vdash R. \end{aligned} \quad (34)$$

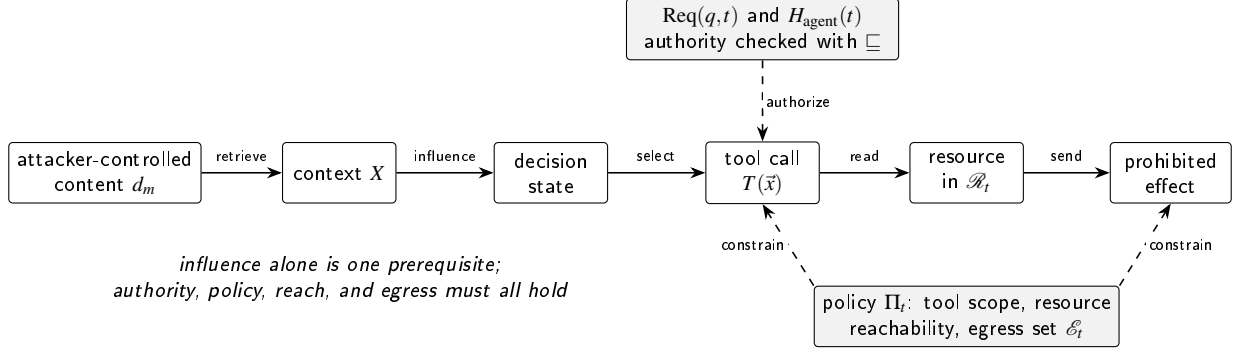


Figure 4: AI-agent extension. Context influence is one prerequisite among several. Parameter-scoped authority, deterministic policy, resource reachability, and egress must all license a path before a prohibited effect is reachable, and each is a point where a control can act.

The agent then acts as executing principal under the attacker’s influence:

$$\begin{aligned}
 ag \triangleleft A \vdash [\text{EXE} \mid \text{INVOKE}] T(\vec{x})@TOOL : \text{INFL}(A, X_{ag}) \wedge \{\text{invoke}(T, \vec{x})\} \subseteq H_t \\
 \wedge \text{ALLOW}(\Pi_t, \text{invoke}(T, \vec{x})) \Rightarrow \text{INVOKED}(T, \vec{x}) : R
 \end{aligned} \tag{35}$$

$$\begin{aligned}
 ag \triangleleft A \vdash [\text{DIS} \mid \text{READ}] \text{search}(s)@TOOL : \text{INVOKED}(\text{search}, s) \wedge \text{secret} \in \mathcal{R}_t(s) \\
 \Rightarrow \text{KNOW}(ag, \text{secret}) : R
 \end{aligned} \tag{36}$$

$$\begin{aligned}
 ag \triangleleft A \vdash [\text{EXF} \mid \text{SEND}] \text{send}(d_A)@EGRESS : \text{KNOW}(ag, \text{secret}) \\
 \wedge \{\text{send}(d_A)\} \subseteq H_t \wedge d_A \in \mathcal{E}_t \wedge \text{ALLOW}(\Pi_t, \text{send}(d_A)) \\
 \Rightarrow \text{POSSESS}(A, \text{secret}) : R
 \end{aligned} \tag{37}$$

The second step is the first under the binding $\theta = \{T \mapsto \text{search}, \vec{x} \mapsto s\}$. The sequence separates context influence, authorized invocation, agent knowledge, and attacker possession. The agent learning a secret does not imply exfiltration; the final effect requires a separately enabled sink transition with parameter-scoped authority, policy permission, and an allowed egress destination. The dual principals record that the agent executed every step and that the attacker influenced it, which is the distinction between influence and authority that recent agent-security work makes central [33, 29].

10.4 Containment as control synthesis

Agent controls act at source admission, context isolation, tool and action allowlists, parameter scope, delegated authority, resource reachability, deterministic policy, and egress. ACN does not require prompt injection to be impossible. It requires the bounded model to have no enabled trace from attacker influence to a prohibited goal while preserving declared task utility, which is Equation (22) applied to agent controls.

11. Falsifiable Hypotheses and Evaluation

Internal consistency is not empirical validation, and no results are reported here. Baselines are unstructured narrative analysis, ATT&CK-only tagging, and an established formal method suited to the experiment, such as MAL, attack-defense trees, or a PDDL planning encoding.

The corpus should cover ransomware, cloud identity compromise, API authorization failure, mobile device takeover, supply-chain compromise, business email compromise, an AI-agent prompt-injection case,

Table 3: *Evaluation program. These are falsifiable hypotheses, not findings.*

Hypothesis	Claim under test	Primary measurements
H1 causal validity	ACN encodings expose seeded missing prerequisites and contradictory effects more reliably than narrative analysis.	Pre-registered defect-detection F1; precision, recall, time.
H2 analyst agreement	Independent analysts produce compatible canonical encodings from the same evidence.	Field-specific Krippendorff α , predicate-set Jaccard, semantic trace comparison.
H3 compiler correctness	Compact to canonical to executable preserves $\mathcal{L}(E, \Sigma_0)$.	Round-trip conformance; property-based tests; counterexample count.
H4 cross-domain adequacy	One core grammar encodes mobile, API, cloud, ransomware, AI, and ICS cases without overloading.	Required extensions; invalid coercions; coverage.
H5 defense quality	Robust task-preserving synthesis reduces prohibited reachability with less task loss than attack-only and single-model baselines.	Residual compromise over $\mathcal{F}_b(D)$; worst-case U_Q ; cost; runtime. Agent instances on AgentDojo [36].
H6 readability	Compact ACN improves comprehension of guards, effects, and evidence at acceptable learning cost.	Accuracy and time; error rate; workload; prior formal-methods experience controlled.

and an ICS process-manipulation case. Each case uses a frozen evidence package, a narrative baseline, external mappings, and a blinded encoding phase. Public reporting, first-party telemetry, and reconstruction must remain distinguishable. Analysts encode independently; primary metrics, sample size, and power analysis are pre-registered; disagreement about evidence status is separated from disagreement about predicate choice. Solver experiments publish the bounded profile: domains, horizon if any, evidence alternatives, model count, bundle count, goals, tasks, exact or approximate method, configuration, and timeout. Because ACN compiles to STRIPS, H5 should include an off-the-shelf planner as a reachability baseline.

ACN should be narrowed or rejected if the core grammar repeatedly needs ad hoc semantics for ordinary cases, analysts cannot reach acceptable agreement after training, the compiler cannot preserve trace semantics, extensions routinely overload predicates, or synthesis shows no benefit over simpler baselines at comparable effort.

12. Conformance Conditions

Requirement 1 (No hidden prerequisite). Every guard atom consumed by a valid trace is present in Σ_0 with an evidence object, or was produced by an earlier transition and not removed before use.

Requirement 2 (Control relevance). A control is relevant to a goal only if its transformation changes the enablement or effects of an executable trace to that goal, changes the initial valuation so the trace becomes invalid, or changes the goal under explicitly modeled recovery semantics.

Requirement 3 (Evidence transparency). Tightening Policy_E to exclude reconstructed hypotheses must not change a conclusion advertised as observed-only. If it does, the conclusion depended on reconstruction and is mislabeled. In modal terms, an observed-only claim of compromise must be a necessary compromise under the observed-only family.

Table 4: *Minimum machine-checkable conformance conditions.*

Check	Requirement
Type correctness	Every value, argument, capability, target, object, channel, and binding satisfies its declared sort.
Ground execution	Executable transitions have no unbound variables; substitutions are type-preserving and consistent.
Explicit values	Guards never use negation-as-failure; negative states are explicit values with provenance.
Guard and invariant validity	Pre holds and the full delete-then-add successor satisfies Γ before enablement.
Effect consistency	$\text{Add} \cap \text{Del} = \emptyset$; valued effects respect exclusivity.
Evidence provenance	Every transition and every initial atom resolves to a source or rationale; confidence has a declared method.
Principals	Executing and influencing principals are distinguished wherever they differ.
Temporal consistency	Ordering does not contradict timestamps or declared causal constraints.
Concurrency	Concurrent transitions satisfy Equation (15) wherever concurrency is claimed.
Control bundles	Each bundle declares an order or proven commutativity and passes conflict validation.
Compiler correctness	Compact to canonical to executable preserves $\mathcal{L}(E, \Sigma_0)$.
Robustness scope	Every containment claim states $\mathcal{F}_b(D)$, goals, bundles, tasks, η , horizon if any, and exact or approximate status.

13. Open Research, Reproducibility, and Release Boundary

This paper is released as independent open-source research, open to implementation, criticism, counterexample construction, and empirical testing. The reference checker published with it implements valued states, invariants, enablement, application, trace execution, independence, exact reachability, possible and necessary compromise, controls, robust synthesis, and subsumption-aware excess authority, and it reproduces every value in Section 8. It is a minimal executable semantics, not the full parser and solver described in Section 15.

The research profile is defensive: incident reconstruction, threat modeling, authorized testing, formal-methods research, education, and control comparison. Public artifacts should model adversary behavior only at the level needed to validate state semantics, without novel exploit code, credential-theft implementations, destructive commands, evasion recipes, or step-by-step intrusion procedures. Synthetic fixtures, like the worked example, are preferred whenever a real artifact would add abuse value without scientific value. This is a release policy, not a claim that a notation can prevent misuse.

14. Limitations

A well-formed model can create false rigor if mistaken for proof about the world; scope and evidence remain explicit for that reason. State explosion is unavoidable as identities, tokens, resources, time, and hypotheses are instantiated, and the complexity bounds of Section 7.2 apply. Boolean and finite-valued atoms are a deliberate simplification: belief, stochastic behavior, continuous processes, and temporal authorization need epistemic, probabilistic, temporal, or hybrid extensions. Intent and mechanism vocabularies drift as external frameworks change, which is why mappings stay metadata. Reconstructed hypotheses remain subject to analyst bias, and the family construction makes that bias visible rather than removing it. The excess-authority measure counts unused capabilities but not over-breadth of used ones. Finally, no multi-analyst study, benchmarked compiler, solver evaluation, or usability experiment is reported; claims of improved

accuracy, usability, or defensive effectiveness remain hypotheses.

15. Roadmap

The next milestone is a public specification package: grammar, type declarations, canonical schema, evidence-policy schema, invariant rules, and conformance tests. The second is a parser and validator that round-trips compact expressions and canonical records and exports ground models to PDDL, so that existing planners serve as reachability engines. The third is an independently encoded corpus with documented adjudication, followed by bounded synthesis experiments. If ACN can map to ATT&CK for taxonomy, STIX for exchange, D3FEND for countermeasures, MAL and planning formalisms for reasoning, and policy engines for enforcement without unacceptable semantic loss, it functions as a bridge rather than another competing encyclopedia. If interoperability or analyst agreement cannot be demonstrated, the project should be narrowed.

16. Conclusion

Attack Calculus does not claim that state machines, precondition-effect attack languages, attack graphs, planning, least privilege, or interdiction are new. Its proposed contribution is their disciplined integration into an analyst-facing intermediate representation: a typed transition that binds principals, intent, mechanism, valued guards, effects, and evidence; a canonical model with non-monotone updates and a known place in the complexity landscape; evidence-consistent families with possible and necessary compromise; and synthesis that eliminates modeled prohibited outcomes in every admissible model while preserving authorized work in the worst admissible model. The worked example shows the two failures this design exists to prevent: a defense that is optimal for the most likely reading of the evidence and wrong for an admissible alternative, and a defense that is cheapest against the attacker and breaks the business. Whether the representation earns its keep is an empirical question, and this release makes it precise enough to test.

Availability

The paper source and the reference checker are published at <https://github.com/codethor0/attack-calculus>. Corrections, counterexamples, and implementations are welcome.

References

- [1] F. Cuppens and R. Ortalo, “LAMBDA: A Language to Model a Database for Detection of Attacks,” in *Recent Advances in Intrusion Detection (RAID)*, LNCS 1907, pp. 197–216, 2000.
- [2] S. J. Templeton and K. Levitt, “A Requires/Provides Model for Computer Attacks,” in *Proc. New Security Paradigms Workshop*, pp. 31–38, 2000.
- [3] S. T. Eckmann, G. Vigna, and R. A. Kemmerer, “STATL: An Attack Language for State-Based Intrusion Detection,” *Journal of Computer Security* 10(1–2), 71–103, 2002.
- [4] P. Ning, Y. Cui, and D. S. Reeves, “Constructing Attack Scenarios through Correlation of Intrusion Alerts,” in *Proc. ACM CCS*, pp. 245–254, 2002.
- [5] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated Generation and Analysis of Attack Graphs,” in *Proc. IEEE Symposium on Security and Privacy*, pp. 273–284, 2002.
- [6] P. Ammann, D. Wijesekera, and S. Kaushik, “Scalable, Graph-Based Network Vulnerability Analysis,” in *Proc. ACM CCS*, pp. 217–224, 2002.

- [7] X. Ou, S. Govindavajhala, and A. W. Appel, “MulVAL: A Logic-based Network Security Analyzer,” in *Proc. USENIX Security Symposium*, 2005.
- [8] B. Kordy, S. Mauw, S. Radomirović, and P. Schweitzer, “Attack-Defense Trees,” *Journal of Logic and Computation* 24(1), 55–87, 2014.
- [9] W. Wideł et al., “The Meta Attack Language: A Formal Description,” *Computers & Security* 130, 103284, 2023.
- [10] A.-M. Konsta, A. Lluch Lafuente, B. Spiga, and N. Dragoni, “Survey: Automatic Generation of Attack Trees and Attack Graphs,” *Computers & Security* 137, 103602, 2024.
- [11] M. Boddy, J. Gohde, T. Haigh, and S. Harp, “Course of Action Generation for Cyber Security Using Classical Planning,” in *Proc. ICAPS*, pp. 12–21, 2005.
- [12] J. Lucangeli Obes, C. Sarraute, and G. Richarte, “Attack Planning in the Real World,” in *Proc. AAAI Workshop on Intelligent Security (SecArt)*, 2010. arXiv:1306.4044.
- [13] J. Hoffmann, “Simulated Penetration Testing: From ‘Dijkstra’ to ‘Turing Test++’,” in *Proc. ICAPS*, 2015.
- [14] R. E. Fikes and N. J. Nilsson, “STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving,” *Artificial Intelligence* 2(3–4), 189–208, 1971.
- [15] D. McDermott et al., “PDDL: The Planning Domain Definition Language,” Technical Report CVC TR-98-003, Yale Center for Computational Vision and Control, 1998.
- [16] T. Bylander, “The Computational Complexity of Propositional STRIPS Planning,” *Artificial Intelligence* 69(1–2), 165–204, 1994.
- [17] H. J. Genrich and K. Lautenbach, “System Modelling with High-Level Petri Nets,” *Theoretical Computer Science* 13(1), 109–135, 1981.
- [18] A. Mazurkiewicz, “Trace Theory,” in *Petri Nets: Applications and Relationships to Other Models of Concurrency*, LNCS 255, pp. 279–324, 1987.
- [19] P. Godefroid, *Partial-Order Methods for the Verification of Concurrent Systems*, LNCS 1032, Springer, 1996.
- [20] MITRE ATT&CK, “Updates: April 2026” (v19) and “Updates: August 2026” (v19.2), The MITRE Corporation, 2026. <https://attack.mitre.org/resources/updates/>
- [21] P. E. Kaloroumakis and M. J. Smith, “Toward a Knowledge Graph of Cybersecurity Countermeasures,” The MITRE Corporation, 2021; MITRE D3FEND. <https://d3fend.mitre.org/>
- [22] OASIS Cyber Threat Intelligence Technical Committee, “STIX Version 2.1,” OASIS Standard, 2021.
- [23] A. K. Nandi, H. R. Medal, and S. Vadlamani, “Interdicting Attack Graphs to Protect Organizations from Cyber Attacks: A Bi-Level Defender-Attacker Model,” *Computers & Operations Research* 75, 118–131, 2016.
- [24] T. H. Bhuiyan, H. R. Medal, A. K. Nandi, and M. Halappanavar, “Risk-Averse Bi-Level Stochastic Network Interdiction Model for Cyber-Security Risk Management,” *International Journal of Critical Infrastructure Protection* 32, 100408, 2021.
- [25] T. Thor, “Mission-Invariant Architecture Morphing: Service-Graph Reconfiguration Against Post-Access Reconnaissance, with Cryptographic Epoch Isolation and Mission-Domain State Continuity,” Zenodo, 2026. <https://doi.org/10.5281/zenodo.23001045>
- [26] OpenAI, “Understanding Prompt Injections: A Frontier Security Challenge,” November 2025.
- [27] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, “Prompt Injection Attack to Tool Selection in LLM Agents,” arXiv:2504.19793, 2025.
- [28] F. Jia, T. Wu, X. Qin, and A. Squicciarini, “The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents,” in *Proc. ACL*, 2025.
- [29] E. Debenedetti et al., “Defeating Prompt Injections by Design,” arXiv:2503.18813, 2025.
- [30] T. Shi et al., “Progent: Programmable Privilege Control for LLM Agents,” arXiv:2504.11703, 2025.
- [31] OpenAI, “Designing AI Agents to Resist Prompt Injection,” March 2026.
- [32] Microsoft Security, “Least Privilege for AI Agents: Identity, Access, and Tool Binding,” Microsoft Security Blog, July 16, 2026.

- [33] P. Wang, Y. Li, and Y. Tian, “Aligning Provenance with Authorization: A Dual-Graph Defense for LLM Agents,” arXiv:2605.26497, 2026.
- [34] W. Xiong, R. Karanjai, Y. Lu, W. Shi, and L. Xu, “Reachability-Based Capability Confinement for LLM Agents under Indirect Prompt Injection,” arXiv:2608.30041, 2026.
- [35] S. Pan, X. Sun, T. Zhang, D. Liao, M. Si, and Z. Xing, “SkillGuard: A Permission Framework for Agent Skills,” arXiv:2606.03024v1, 2026. (Revised in v2 as “SkillGuard: A Permission-Centric Framework for Agent Skill Security.”)
- [36] E. Debenedetti et al., “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” arXiv:2406.13352, 2024.
- [37] W3C, “PROV-DM: The PROV Data Model,” W3C Recommendation, 2013.
- [38] ThreatFabric Mobile Threat Intelligence, “Manic: Blend between Banking Malware and Spyware,” August 20, 2026.

Appendix A. Core Grammar and Precedence

$$\begin{aligned}
 \textit{Atomic} &::= \textit{principal} [\triangleleft \textit{principal}] \vdash [\textit{intent} \mid \textit{mechanism}] \textit{target}(\textit{object}) @ \textit{channel} \\
 &: \textit{Pre} \Rightarrow (\textit{Add}, \textit{Del}) \dot{:} \textit{evidence} \\
 \textit{Expr} &::= \textit{Choice} \\
 \textit{Choice} &::= \textit{Parallel} \{ \oplus \textit{Parallel} \} \\
 \textit{Parallel} &::= \textit{Sequence} \{ \parallel \textit{Sequence} \} \\
 \textit{Sequence} &::= \textit{Repeat} \{ \rightarrow \textit{Repeat} \} \\
 \textit{Repeat} &::= \textit{Primary} [* \mid \leq^k] \\
 \textit{Primary} &::= \not\vdash \mid \textit{Atomic} \mid (\textit{Expr})
 \end{aligned}$$

Parentheses bind explicitly; repetition binds most tightly, then sequence, then concurrency, then choice. Star is finite Kleene closure; $\leq^k$ is bounded repetition. Executable transitions are ground after type-preserving substitution.

Appendix B. Canonical Record Example

```

transition_id: api-bola-read-001
executing_principal: A
influencing_principal: null
intent: COL
mechanism: ACCESS
target: API
object: record_j
channel: HTTPS
preconditions:
  - AUTHN(A)=true
  - AUTHZ(A,j)=denied
  - ENFORCED(API,j)=false
add_effects:
  - KNOW(A,record_j)=true
delete_effects: []
evidence:
  status: R
  confidence: null

```

```

method: null
provenance: synthetic-example
support: [guard, add_effect]
initial_atom_evidence:
- atom: ENFORCED(API,j)=false
  status: R
  provenance: synthetic-example
framework_mappings: []

```

Appendix C. Notation

Symbol	Meaning
Σ, Σ_0	Well-typed state; initial state
Γ	Domain invariants, including value exclusivity, checked on the full successor
Δ	Type environment
$\tau; a \triangleleft b$	Ground transition; executing principal a influenced by b
Pre, Add, Del	Valued guards; additive and destructive effects
e	Evidence object (transitions and initial atoms)
$\mathbb{K}; \rightarrow, \parallel, \oplus, *, \leq^k$	Identity; sequence, concurrency, choice, repetition, bounded repetition
$\mathcal{L}(E, \Sigma)$	Finite trace language of E from Σ
$\mathcal{M}$	Guarded predicate-transition model
$\mathcal{F}(D), \mathcal{F}_b(D)$	Evidence-consistent family; bounded family
$G, g; Q, q$	Prohibited goals; authorized tasks
$B, J(B), \eta$	Control bundle, cost, utility floor
$X, H, \mathcal{O}, \mathcal{R}, \Pi, \mathcal{E}$	Agent context, capabilities, tool operations, resources, policy, permitted effects
$c \sqsubseteq c'$	Capability subsumption
$EA_{\min}, EA_{\max}$	Excess-authority interval